Jorge Gael Hernández Marcial

Ejercicios

1. Vocabulario controlado. Haz que POST y PATCH solo acepten estado con valores abierta, en_proceso o resuelta; cualquier otro valor debe responder 400 con un mensaje que liste los valores válidos.



2. La libreta en disco. Crea dos incidencias, abre incidencias.json y verifica que están escritas. Detén el servidor (Ctrl+C), vuelve a arrancarlo y pide GET /incidencias: deben seguir ahí, con el siguiente id continuando donde se quedó. ¿Por qué /logs y /contadores sí vuelven a cero? Explícalo en el README.



las incidencias persisten al reiniciar el servidor porque se respaldan en el disco duro mediante la función guardar(), la cual utiliza json.dump() para reescribir el archivo físico incidencias.json tras cada modificación. Por el contrario, las variables logs y contadores se reinician a cero porque son estructuras de datos globales en Python que residen exclusivamente en la memoria RAM y se borran al detener el script. Respecto al método PUT no implementado inicialmente, el servidor arroja un código 501 porque la clase base heredada BaseHTTPRequestHandler intercepta la petición al no encontrar una función do_PUT explícita. Como esta respuesta automática generada por la biblioteca no pasa por nuestra función personalizada _enviar(), la petición jamás entra a la lógica que actualiza la bitácora ni los contadores.

3. El 501 invisible. Ejecuta curl -i -X PUT http://localhost:8001/incidencias/1 y luego pide /contadores y /logs. El PUT aparece en la terminal del servidor con código 501,

pero no está ni en los contadores ni en la bitácora. ¿Por qué? Pista: ¿quién genera esa respuesta?



El error 501 lo genera la clase base BaseHTTPRequestHandler al no encontrar el método. Como la petición nunca entra a nuestra función _enviar(), no se registra en las métricas.

4. Implementa do_PUT. Reemplazo completo: el id llega en la ruta y el cuerpo debe traer tipo y descripcion (si falta alguno, 400). Conserva el id original, reinicia estado a abierta, llama a guardar() y responde 200 con el registro. Verifica que ahora sí aparece en /contadores y /logs, y en el archivo.



Reinicia el servidor y prueba el reemplazo completo. Verifica que el estado se reinicia a "abierta" automáticamente

5. Filtros combinados. Agrega el parámetro estado a GET /incidencias y haz que pueda combinarse con tipo: /incidencias?tipo=alerta&estado=abierta. Prueba todas las combinaciones con curl y con el cliente en Python.



Lo que pasó fue que en el paso anterior usé el método PUT para hacer un reemplazo completo del registro 1, y ahí le cambié el valor de "tipo" a "vulnerabilidad". Entonces, cuando le lanzo la petición GET pasándole por la URL los parámetros ?tipo=alerta&estado=abierta, mi código en la función do_GET recorre los datos buscando una incidencia que cumpla obligatoriamente con ambas condiciones al mismo tiempo.

Como el único registro que tengo guardado ahorita en incidencias.json es una "vulnerabilidad", el filtro de "alerta" ya no hace *match* con nada. Al no encontrar ninguna coincidencia, el servidor hace lo correcto y me regresa la lista vacía [] en

formato JSON, lo que demuestra que el código está filtrando la información a la perfección.